

Exploitation d'une BD – TP4

Corrigé TP4 Auto-jointure

L'auto-jointure se produit lorsqu'une table est jointe à elle-même. Cela peut être utile pour la comparaison de valeurs au sein de la même table ou l'analyse de relations hiérarchiques.

Syntaxe.

```
SELECT ...
FROM table t1 JOIN table t2
ON condition ;
```

Exercice 1.

Considérons la table « **Contenu** » suivante :

On cherche les **paires** de plats qui ont au moins un ingrédient en commun. On aimerait donc construire une requête retournant ceci :

<i>plat_1</i>	<i>plat_2</i>
1	2
1	3

Contenu

<i>plat</i>	<i>ingrédient</i>
1	Farine
1	Œuf
2	Farine
2	Beurre
3	Œuf
3	Sucre

1. Pourquoi y a-t-il la ligne (1, 3) ?

Car 3 et 1 sont deux plats qui utilisent de l'Oeuf.

2. Pourquoi n'y a-t-il pas de ligne (2, 3) ?

Car 2 et 3 n'ont aucun ingrédient en commun dans leur recette.

3. Construisez une requête retournant tous les **couples** de plats qui ont au moins un ingrédient en commun :

```
SELECT C1.plat plat_1, C2.plat plat_2
FROM Contenu C1 INNER JOIN contenu C2
ON C1.ingrédient=C2.ingrédient
```

<i>plat_1</i>	<i>plat_2</i>
1	1
1	2
1	3
2	1
2	2
3	1
3	3

4. a. En déduire la requête retournant ceci :

```
SELECT C1.plat plat_1, C2.plat plat_2
FROM Contenu C1 INNER JOIN contenu C2
ON C1.ingrédient = C2.ingrédient
WHERE C1.plat <> C2.plat
```

<i>plat_1</i>	<i>plat_2</i>
1	2
1	3
2	1
3	1

- b. Nous voudrions supprimer les lignes (2, 1) et (3, 1) car elles n'apportent aucune information nouvelle par rapport à (1, 2) et (1, 3). Construisez la requête retournant ceci :

```
SELECT C1.plat plat_1, C2.plat plat_2
FROM Contenu C1 INNER JOIN contenu C2
ON C1.ingredient=C2.ingredient
WHERE C1.plat<C2.plat
```

plat_1	plat_2
1	2
1	3

5. On s'intéresse à la requête suivante :

```
SELECT C1.ingredient ingredient_1, C2.ingredient ingredient_2
FROM Contenu C1 INNER JOIN Contenu C2
ON C1.plat = C2.plat ;
```

Complétez le tableau ci-contre listant les 10 lignes retournées par cette requête.

ingredient_1	ingredient_2
farine	farine
farine	œuf
farine	beurre
œuf	farine
œuf	œuf
œuf	sucre
beurre	farine
beurre	beurre
sucre	œuf
sucre	sucre

ingredient_1	ingredient_2
farine	
farine	
farine	
œuf	
œuf	
œuf	
beurre	
beurre	
sucre	
sucre	

6. Comment définiriez-vous, à l'aide d'une phrase, ce que nous retournerait cette requête ?

```
SELECT C1.ingredient ingredient_1, C2.ingredient ingredient_2
FROM Contenu C1 INNER JOIN Contenu C2
ON C1.plat = C2.plat
WHERE C1.ingredient<>C2.ingredient
```

Cette requête retourne les paires d'ingrédients différents qui figurent dans une même recette.

Exercice 2.

Considérons une table **Employes** où l'attribut *idSuperieur* fait référence à l'identifiant du responsable d'un employé.

idEmploye	nom	prenom	salaire	idSuperieur
1	Dupont	Alice	2500	NULL
2	Martin	Bob	4300	1
3	Lefevre	Caroline	4800	1
4	Roux	David	2100	2

Foreign key

5	Gagnon	Eva	3600	2
6	Leclerc	Francois	NULL	3
7	Cote	Gisele	3100	3
8	Tremblay	Henri	NULL	3

- Donner les requêtes de création et d'insertion de données dans cette table.

```
CREATE TABLE Employes (
    idEmploye INT PRIMARY KEY,
    nom VARCHAR(50) NOT NULL,
    prenom VARCHAR(50) NOT NULL,
    salaire DECIMAL(10, 2),
    idSuperieur INT,
    FOREIGN KEY (idSuperieur) REFERENCES Employes(idEmploye)
);
```

```
INSERT INTO Employes VALUES
(1, 'Dupont', 'Alice', 2500, NULL),
(2, 'Martin', 'Bob', 4300, 1),
(3, 'Lefevre', 'Caroline', 4800, 1),
(4, 'Roux', 'David', 2100, 2),
(5, 'Gagnon', 'Eva', 3600, 2),
(6, 'Leclerc', 'Francois', NULL, 3),
(7, 'Cote', 'Gisele', 3100, 3),
(8, 'Tremblay', 'Henri', NULL, 3);
```

- Écrivez une requête pour afficher le nom, le prénom et le nom du supérieur de tous les employés. Afficher « Sans supérieur » pour les employés n'ayant pas de supérieur.

```
select e.nom AS "Nom employé", e.prenom AS "Prénom employé",
       ifnull(s.nom, "Sans supérieur") AS "Nom du supérieur"
from Employes e left join Employes s
on e.idSuperieur=s.idEmploye ;
```

- Écrivez une requête pour lister les employés qui ne sont supérieurs d'aucun autre employé.

```
## Requête avec not NULL
select *
from Employes
where idEmploye not in (select idSuperieur from Employes where idSuperieur is not null ) ;
```

```
select e.*
from Employes e left join Employes s
on e.idEmploye=s.idSuperieur
WHERE s.idEmploye is null ;
```

- Écrivez une requête permettant d'afficher, pour chaque employé, son nom, son prénom, le salaire moyen de ses subordonnés ainsi que le nombre d'employés qu'il supervise. Pour les employés ne supervisant personne, le salaire moyen devra être à 0.

Aperçu attendu :

nom	prenom	Nombre d'employés supervisés	Salaire moyen des employés
Dupont	Alice	2	4550
Martin	Bob	2	2850
Lefevre	Caroline	3	3100
Roux	David	0	0
Gagnon	Eva	0	0

```
select e.nom, e.prenom, COUNT(f.idEmploye) AS "Nombre d'employés supervisés",
       IFNULL(AVG(f.salaire), 0) AS "Salaire moyen des employés"
from Employes e left join Employes f
on e.idEmploye=f.idSuperieur
group by e.idEmploye ;
```

Exercice 3.

Reprenons la base de données **EDTS2** qui a le même schéma que la base de base de données EDT.

- Ecrivez la requête permettant d'afficher les couples d'enseignants qui enseignent les mêmes modules.

Aperçu attendu :

Enseignant1	Module1	Enseignant2	Module2
LMI	M4101C	LPO	M4101C
ABE	M3303	JBO	M3303
LBU	M1102	LMO	M1102
ADA	M3206	HLA	M3206
APE	M3103	VOS	M3103

```
SELECT DISTINCT s1.idProf AS Enseignant1, s1.idModule AS Module1,
                s2.idProf AS Enseignant2, s2.idModule AS Module2
FROM Seances s1 JOIN Seances s2
ON s1.idModule = s2.idModule
WHERE s1.idProf < s2.idProf ;
```

- Ecrivez la requête permettant d'afficher les demi-groupes qui ont le même nombre d'étudiants.

Aperçu attendu :

sousGroupe1	effectifGroupe1	sousGroupe2	effectifGroupe2
S1G1.1	13	S1G1.2	13
S1G1.1	13	S1G2.2	13
S1G1.2	13	S1G2.2	13
S1G1.1	13	S1G3.1	13
S1G1.2	13	S1G3.1	13

```

select g1.idGroupe AS sousGroupe1, g1.effectif AS effectifGroupe1,
       g2.idGroupe AS sousGroupe2, g2.effectif AS effectifGroupe2
from Groupes g1 JOIN Groupes g2 ON g1.effectif = g2.effectif
WHERE g1.idGroupe LIKE "S_G_._" AND g2.idGroupe LIKE "S_G_._"
      AND g1.idGroupe < g2.idGroupe ;

```

3. Trouver les enseignants qui utilisent les mêmes salles.

Aperçu attendu :

	Enseignant1	Salle1	Enseignant2	Salle2
▶	ABE	Amphithéâtre Le Bugey	FVA	Amphithéâtre Le Bugey
	ABE	Amphithéâtre Le Bugey	VOS	Amphithéâtre Le Bugey
	ABE	Amphithéâtre Le Bugey	KBE	Amphithéâtre Le Bugey
	ABE	Amphithéâtre Le Bugey	FJA	Amphithéâtre Le Bugey
	ABE	Amphithéâtre Le Bugey	JFA	Amphithéâtre Le Bugey

```

SELECT distinct s1.idProf AS Enseignant1, u1.idSalle AS Salle1, s2.idProf AS Enseignant2, u2.idSalle AS Salle2
FROM Seances s1, Seances s2, Utilise u1, Utilise u2
where s1.idSeance=u1.idSeance and s2.idSeance=u2.idSeance AND u1.idSalle = u2.idSalle
      and s1.idProf < s2.idProf ;

```

Avec JOIN

```

SELECT distinct s1.idProf AS "Enseignant_1", u1.idSalle AS "Salle_1",
               s2.idProf AS "Enseignant_2", u2.idSalle AS "Salle_2"
FROM Seances s1 JOIN Utilise u1 ON s1.idSeance=u1.idSeance,
     Seances s2 JOIN Utilise u2 ON s2.idSeance=u2.idSeance
WHERE u1.idSalle = u2.idSalle AND s1.idProf < s2.idProf ;

```

4. Pour chaque couple d'enseignants, dites combien de fois ils ont utilisé une même salle.

Aperçu attendu :

	Enseignant1	Enseignant2	Nombre de salles communes
▶	ABE	ABI	6
	ABE	ADA	15
	ABE	ADU	9
	ABE	AEP	1
	ABE	APE	12

```

SELECT s1.idProf AS Enseignant1, s2.idProf AS Enseignant2 ,
       COUNT(distinct u2.idSalle) AS "Nombre de salles communes"
FROM Seances s1, Seances s2, Utilise u1, Utilise u2
where s1.idSeance=u1.idSeance and s2.idSeance=u2.idSeance AND u1.idSalle = u2.idSalle
      and s1.idProf < s2.idProf
group by s1.idProf, s2.idProf ;

```

5. S1G1.1 a pour parent S1G1. S1G1 a lui-même pour parent S1. On peut donc dire que S1G1.1 a pour grand_parent S1. Ecrivez la requête précisant, pour chaque groupe ayant un grand-parent, le nom du groupe et le nom de son grand-parent. Vous utiliserez une auto-jointure. (30 lignes retournées)

groupe	a pour grand parent
S1G1	Sans
S1G2	Sans
S1G3	Sans
S1G1.1	S1
S1G1.2	S1
S1G2.1	S1

```
select g1.idGroupe groupe, IFNULL(g2.parent, "Sans") "a pour grand parent"
from Groupes g1 join Groupes g2
on g1.parent=g2.idgroupe ;
```

6. S1G1.1 est inclus dans S1G1. S1G1.1 est aussi inclus dans S1. Ecrivez la requête précisant toutes les inclusions strictes qui existent ("strictes" de manière à ne pas tenir compte du fait qu'un groupe est toujours inclus dans lui-même). (75 lignes retournées)

Aperçu attendu :

idGroupe	est inclus dans
S4G3.2	S4
S5G3.1	S4
S5G3.2	S4
S1G1	S1
S1G2	S1
S1G3	S1
S1G1.1	S1G1
S1G1.2	S1G1
S1G2.1	S1G2

```
SELECT g.idGroupe as "Groupe", h.parent as "Inclus dans "
FROM Groupes g JOIN Groupes h
on g.parent=h.idGroupe
where h.parent is not null
UNION
SELECT idGroupe as "Groupe", parent as "Inclus dans "
FROM Groupes
where parent is not null ;
```